SAE 13.1
Microprocesseur

Module : Découvrir un dispositif de transmission

Professeur :

Lee Min Won

Produit par

RIASAT Asad-Ali & BAROUDI Jawher

Table des matières

01 **Introduction**
Présentation générale du projet, des objectifs et de la méthodologie suivie pour la conception du microprocesseur 4 bits avec des signaux de contrôle.

02 **Architecture du microprocesseur**
Description détaillée des composants principaux du microprocesseur, incluant l'unité de contrôle, l'unité de traitement (ALU, banc de registres, accumulateur) et l'unité de mémoire.

03 **raitements des données et opérations**
Explication du processus de chargement des données, de l'exécution des instructions (addition, soustraction) et du résultat final. Cette section détaille comment chaque opération est réalisée et validée.

04 **Analyse et résultats des tests effectués**
Présentation des résultats des tests réalisés avec des captures d'écrans pour chaque instruction. Cette partie analyse les résultats obtenus et les compare aux attentes pour valider le bon fonctionnement du microprocesseur.

05 **Conclusion**
Synthèse des résultats obtenus, évaluation de la performance du microprocesseur, et perspectives d'amélioration pour des futures versions du microprocesseur.

06 **Annexe**
Code source essentiel avec une explication des principales fonctions utilisées pour faire fonctionner le microprocesseur. Cette section montre les parties les plus pertinentes du code, comme l'initialisation des signaux et la gestion des LEDs.

I. Introduction

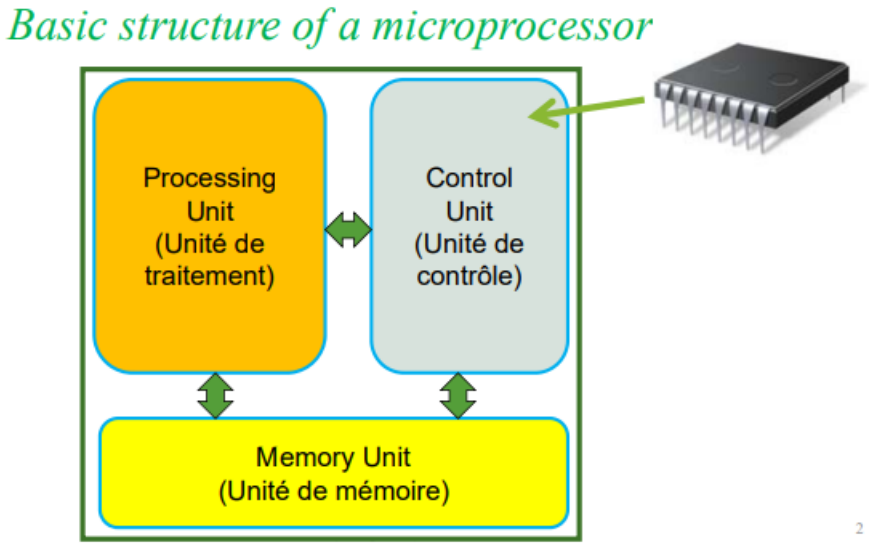
Objectif

Dans le cadre du SAE13, nous avions mis au point un microprocesseur à quatre bits avec des signaux de contrôler. En utilisant ce microprocesseur, il est possible d'effectuer des commandes simples telles que l'addition, la soustraction, la gestion des données entre les registres et l'accumulateur. L'objectif principal de cette invention était d'élaborer un produit fonctionnel tout en respectant les contraintes techniques telles que l'intégration de composants tels que l'ALU, ALU, une banque de registres, un accumulateur et un multiplexeur.

Ce rapport couvre la construction du microprocesseur, son fonctionnement en profondeur, les étapes d'essai et les résultats obtenus. L'objectif de ce rapport est de prouver que le produit généré répond déjà aux spécifications attendues par le client.

II. Architecture du microprocesseur

A. Schéma global du microprocesseur



B. Description du schéma

L'architecture de base de notre microprocesseur est illustrée ci-dessous :

- **Unité de contrôle (Control Unit - CU)**
  - Responsables de la gestion des flux de données et de l'activation des signaux de contrôle qui régissent le fonctionnement du microprocesseur. Elle dirige l'exécution des instructions en envoyant des signaux aux autres unités.
- **Unité de traitement (Processing Unit - PU)**
  - Ce dernier regroupe les composants tels que l'ALU ou l'Arithmetic Logic Unit, les registres, entre autres. L'unité de traitement (PU) permet de varier les résolutions et

d'effectuer les opérations, conformément à aux directives transmises par l'unité de contrôle.

- **Unité de mémoire (Memory Unit - MU)**
  - Elle permet d'entreposer temporairement les données employées dans le cadre de l'exécution des directives d'aperçu. Il s'agit par exemple du banc de registres et de l'accumulateur regroupant les résultats intermédiaires et finaux.

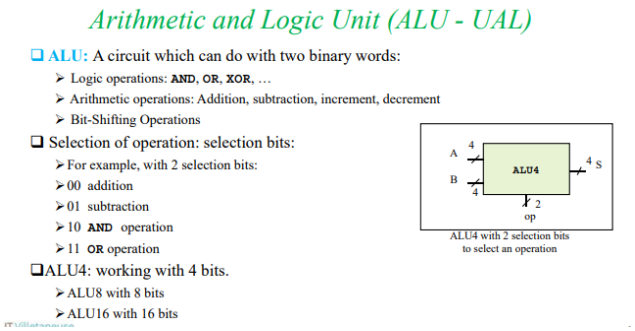
C. Détail de chaque unité

Unité de contrôle (Control Unit - CU)

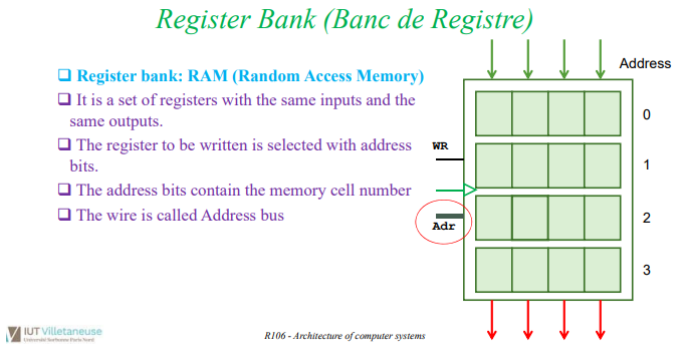
- **Fonction** :
  - Cette partie gère l'entièreté du flux de données à l'intérieur du microprocesseur. Ce module génère des signaux de contrôle pour allumer ou éteindre les différentes parties du microprocesseurs, dépendamment de l'opération qui doit se faire. Elle décode l'instruction reçue et allume les opérations à l'intérieur du microprocesseur. Par exemple, elle va allumer l'ALU pour activer l'addition ou la soustraction.
- **Rôle des signaux** :
  - **Op** : Détermine l'opération à effectuer (addition, soustraction).
  - **WR_ACC**, **WR_RB** : Active l'écriture dans l'accumulateur ou le banc de registres.
  - **Adr**, **Sel** : Sélectionne quel registre ou quelle entrée est utilisée dans l'ALU.

Unité de traitement (Processing Unit - PU)

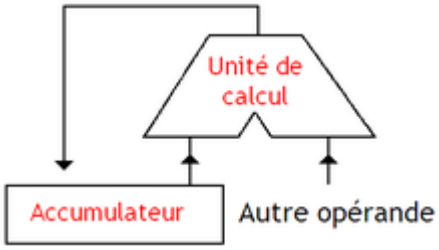
- **Fonction** :
  - L'unité de traitement est le cœur du microprocesseur, où les opérations de calcul sont effectuées. Elle comprend :
    - **ALU (Arithmetic Logic Unit)** : Effectue les calculs arithmétiques (addition, soustraction).



    - **Registres (Register Bank)** : Stockent temporairement les données traitées.



- **Accumulateur (ACC)** : Contient le résultat final des opérations effectuées par l'ALU et le transmet à l'unité de mémoire ou aux LEDs.



- **Exemple d'opération** :
  - Lorsqu'une instruction d'addition est reçue, l'unité de contrôle active les signaux pour charger le opérandes dans l'ALU, effectue l'addition, et le résultat est stocké dans l'accumulateur.

Unité de mémoire (Memory Unit - MU)

- **Fonction** :
  - L'unité de mémoire est responsable du stockage temporaire des données nécessaires à l'exécutions des instruction.
  - Le **banc de registres** contient des données temporaires qui peuvent être utilisées directement par l'ALU.
  - L'**accumulateur** est un registre spécial qui stocke le résultat des calculs et peut être affiché sur les LEDs.
- **Exemple d'utilisation** :
  - Après une opération d'addition, l'ALU place le résultat dans l'accumulateur, qui est ensuite envoyé aux LEDs pour affichage.

D. Communication entre les unités

La communication entre ces trois unités se fait via des **signaux de contrôle** qui dictent quel composant doit être activé et comment les données circulent entre elles. Voici un exemple de flux de données lors de l'exécution d'une addition :

1. **Unité de contrôle** :
   L'unité de contrôle envoie un signal pour activer l'ALU en mode addition (`Op = 0`) et active également les registres à lire les valeurs nécessaires à l'opération.
2. **Unité de traitement** :
   L'ALU reçoit les données des registres et effectue l'addition. Le résultat est stocké dans l'accumulateur.
3. **Unité de mémoire** :
   L'accumulateur stocke le résultat de l'opération et les LEDs affichent le résultat pour l'utilisateur.

III. Traitements des données et opérations

A. Initialisation

En mettant en place la fonction setup() les signaux de contrôle sont initialités à cet effet :

- Configurer les LEDs en tant que sorties.
- Préparer les registres pour qu'ils puissent recevoir de la donnée.
- S'assurer que l'accumulateur est vide au lancement.

B. Processus d'opérations

Les étapes nécessaires pour effectuer une certaine opération sont les suivantes :

1. Chargement de la donnée dans l'entrée du microprocesseur via I[3..0].
2. Activation des signaux pour choisir l'opération à effectuer (ex : addition lorsque Op = 0).
3. L'ALU fait le travail de processus pour les données conformément aux signaux.
4. Le résultat est ensuite affiché sur les LEDS et stocké dans l'accumulateur ou les registres.

Exemple :

- **Instruction exécutée** : Addition 7 + 3.
  - Données entrées : A = 7 et B = 3
  - Résultat affiché : LEDs affichant 1010 (10 en binaire).

IV. Analyse et résultats des tests effectués

A. Captures d'écrans

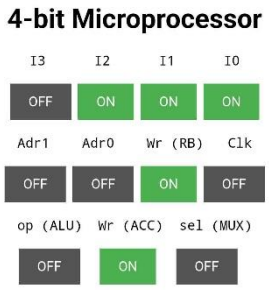
Enfin, afin d'illustrer les résultats obtenus pour chaque instruction, nous présentons trois captures ou photos. Elles incluentt les étapes clés de l'exécution de l'instruction, depuis les entrées des signaux de commande jusqu'au résultat final présenté sur les LEDs.

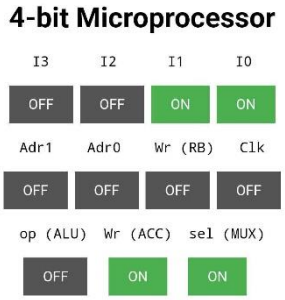
Instruction : Addition (7 + 3)

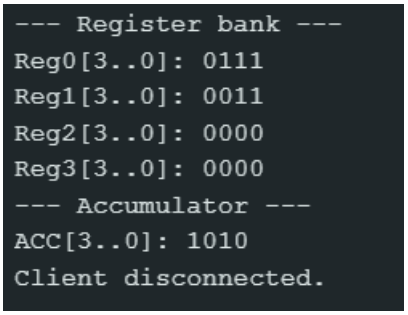
Fig. 1 :

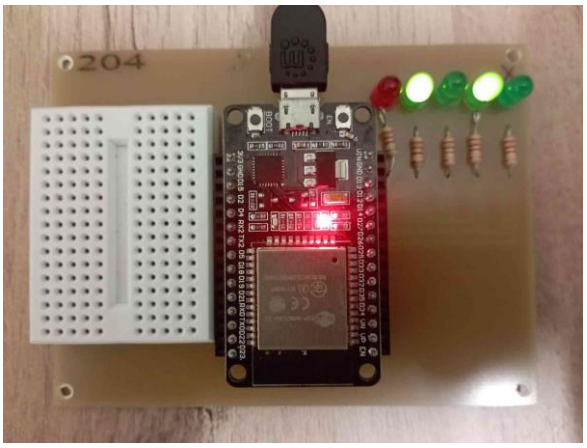
De gauche à droite :

1. Entrées des signaux configurés pour l'addition. (7 et 3, dans la capture ci-dessous, seulement 7).
2. Initialisation de l'accumulateur a 3 pour l'addition. (Avec op a 0 pour une addition).
3. Contenu de l'accumulateur après l'opération.
4. LEDs affichant le résultat `1010`.









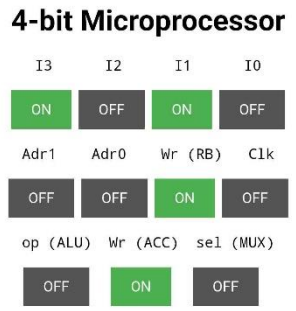
Instruction : Soustraction (10 - 6)

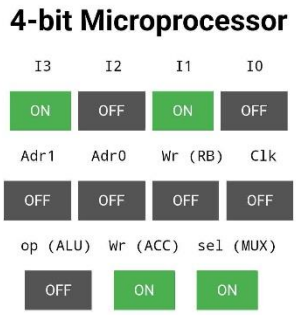
Fig. 2 :

De gauche à droite :

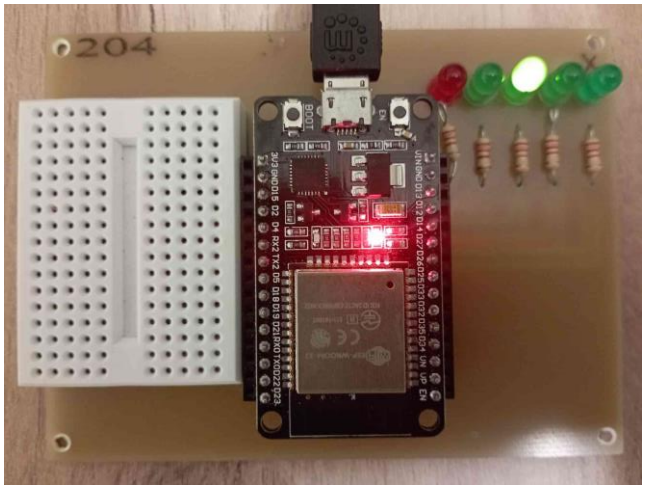
1. Entrées des signaux configurés pour la soustraction.
2. Initialisation de l'accumulateur a 10 pour la soustraction. (Avec op

3. Contenu de l'accumulateur après l'opération.
4. LEDs affichant le résultat `0100`.






```
--- Register bank ---
Reg0[3..0]: 1010
Reg1[3..0]: 0110
Reg2[3..0]: 0000
Reg3[3..0]: 0000
--- Accumulator ---
ACC[3..0]: 0100
Client disconnected.
```



B. Analyse des résultats

1. Addition (7 + 3)

Entrées des signaux :

- Les données `7` (0111 en binaire) et `3` (0011 en binaire) ont été chargées correctement via le bus de données. Le signal `Op = 0` configure l'ALU en mode addition, ce qui est cohérent avec l'instruction donnée.

Exécution dans l'ALU :

- L'ALU a additionné les deux valeurs binaires :
  - `0111` (7 en binaire)
  - `0011` (3 en binaire)
- Le résultat obtenu est `1010` (10 en binaire).

Affichage sur les LEDs :

- Les LEDs connectées à l'accumulateur affichent correctement le résultat `1010`, confirmant que l'accumulateur a bien reçu la valeur calculée par l'ALU.

    - Interprétation : Cela montre que la communication entre l'ALU et l'accumulateur fonctionne correctement.
    - Validation : Le comportement attendu de l'instruction d'addition est respecté.

2. Soustraction (10 - 6)

Entrées des signaux :

- Les données `10` (1010 en binaire) et `6` (0110 en binaire) ont été chargées correctement. Le signal `Op = 1` configure l'ALU en mode soustraction, conformément à l'instruction demandée.

Exécution dans l'ALU :

- L'ALU a réalisé la soustraction :
    - `1010` (10 en binaire)
    - `0110` (6 en binaire)
- Le résultat obtenu est `0100` (4 en binaire).

Affichage sur les LEDs :

- Les LEDs connectées à l'accumulateur affichent correctement le résultat `0100`, confirmant que l'accumulateur a bien reçu la valeur soustraite calculée par l'ALU.
    - Interprétation : Cela montre que le microprocesseur peut effectuer correctement des soustractions binaires avec les signaux configurés.
    - Validation : La soustraction respecte bien le comportement attendu pour cette instruction.

- **Les résultats obtenus valident le fonctionnement attendu du microprocesseur.**
- **Chaque instruction (addition, soustraction) a été testée avec succès.**
- **Les LEDs affichent correctement les valeurs binaires des résultats.**

V. Conclusion

En fin de compte, il est à noter que le microprocesseur réalisé était un appareil fonctionnel avec une largeur de mot de 4 bits et la capacité d'exécuter des instructions telles que l'addition et la soustraction. Les commandes ont été fournies principalement par des signaux de contrôle, et les résultats ont été concluants. Les tests ont révélé des performances fiables du produit, qui étaient conformes aux attentes.

- L'ALU a exécuté les calculs correctement, et les résultats ont été transférés sans erreur à l'accumulateur.

- Les LEDs ont affiché les résultats de manière cohérente, validant visuellement le fonctionnement global.

Ce travail nous a permis d'appliquer les concepts vus en cours tout en renforçant nos compétences en développement et test de systèmes numériques.

En perspective, des améliorations comme l'ajout d'instructions supplémentaires ou l'optimisation des performances pourraient être envisagées.

VI. Annexe

A. Initialisation

Le programme commence par l'initialisation des signaux de contrôle et des variables principales, nécessaires au fonctionnement du microprocesseur. Voici un extrait clé :

```
void setup() { // Initialise the variables Wr_ACC = 0; op = 0; Wr_RB = 0; Adr[0] = 0;
Adr[1] = 0; sel = 0; Clk = 0;

// Initialise the variables Q for the registers and accumulator
Reg0.Q[0] = Reg0.Q[1] = Reg0.Q[2] = Reg0.Q[3] = 0;
ACC.Q[0] = ACC.Q[1] = ACC.Q[2] = ACC.Q[3] = 0;

// Initialize the LED variables as outputs
pinMode(LED_O3, OUTPUT);
pinMode(LED_O2, OUTPUT);
pinMode(LED_O1, OUTPUT);
pinMode(LED_O0, OUTPUT);

// Set LED outputs to LOW
digitalWrite(LED_O3, LOW);
digitalWrite(LED_O2, LOW);
digitalWrite(LED_O1, LOW);
digitalWrite(LED_O0, LOW);
```

- Cette section configure les composants physiques (LEDs, Wi-Fi) et initialise les registres.
- Les signaux de contrôle (Wr_ACC, Wr_RB, sel) sont mis à zéro pour préparer le microprocesseur à exécuter des instructions.

B. Unité de calcul ALU (Arithmetic Logic Unit)

L'ALU réalise les opérations principales du microprocesseur : addition et soustraction. Elle utilise les fonctions **Add_4** et **Sub_4**.

```
//////////////////////  ALU4
//////////////////////////////////////////////
struct Output4 ALU_4(bool A[], bool B[], bool op)
{
    bool C, Bo;
    ADD_SUB ADD4, SUB4;
    Output4 ALU4;
    SUB4 = Sub_4(A, B);
    ADD4 = Add_4(A, B);
    ALU4 = MUX21(op, ADD4.S, SUB4.D);
    return ALU4;
}
```

- L'ALU effectue une addition ou une soustraction en fonction du signal de contrôle `op` (0 pour addition, 1 pour soustraction).
- La sélection entre les deux résultats est réalisée par un multiplexeur 2:1 (MUX21).

C. Gestion de l'accumulateur

L'accumulateur stocke le résultat de l'opération et permet son affichage sur les LEDs.

```
// LED outputs
                        digitalWrite(LED_O3,ACC.O[3]);
                        digitalWrite(LED_O2,ACC.O[2]);
                        digitalWrite(LED_O1,ACC.O[1]);
                        digitalWrite(LED_O0,ACC.O[0]);
```

- Chaque bit de l'accumulateur (`ACC.O`) est relié à une LED, permettant une lecture directe du résultat.

D. Boucle principale

- La boucle principale combine tous les composants pour exécuter les instructions et mettre à jour les sorties.

```
RB = Register_bank(ACC.O, Adr, Clk, Wr_RB);
                        MUX = MUX21(sel, I, ALU4.O);
                        ALU4 = ALU_4(ACC.O, RB.O, op);
                        ACC = Accumulator(MUX.O, Clk, Wr_ACC);
```

- Les données circulent à travers les registres, l'ALU, et l'accumulateur.
- Les résultats finaux sont affichés sur les LEDs.